Tending to Troy

Technical Architecture & Engineering Specification

Version 1.0

Purpose

This document describes the preferred technical architecture for Tending to Troy.

The objective is to create a fast, maintainable and elegant application that can be comfortably maintained by AI-assisted development over many years.

The project should favour simplicity over unnecessary complexity.

Avoid overengineering.

Core Principles

The application should be:

Fast.

Simple.

Reliable.

Maintainable.

Secure.

Responsive.

Offline-friendly where practical.

Cross-platform.

Preferred Platform

Primary platform:

Progressive Web Application (PWA)

Reasons:

Single codebase

Works on iPhone

Works on Android

Desktop compatible

Easy deployment

Instant updates

No App Store approval required

Future native applications can be produced later if desired.

Recommended Technology Stack

Frontend

Next.js

React

TypeScript

Tailwind CSS

Backend

Supabase

Database

PostgreSQL (via Supabase)

Authentication

Supabase Auth

Hosting

Vercel

Weather

UK Met Office Data API

Future Integrations

Monzo API

Retailer APIs

AI services

Application Architecture

Client

↓

Authentication

↓

Supabase API

↓

PostgreSQL Database

↓

Real-time Synchronisation

↓

Both Users

All updates should synchronise automatically.

Authentication

Email

Password

Invitation-only household

Version 1 supports exactly one household.

Exactly two users.

Future architecture should allow multiple households without requiring database redesign.

Database Design

Household

id

name

created_at

Users

id

household_id

name

email

created_at

Tasks

id

household_id

title

priority

manual_order

owner

location

status

description

notes

is_pinned

created_by

created_at

updated_at

completed_at

archived_at

Checklist Items

id

task_id

text

completed

manual_order

Shopping Items

id

task_id

name

quantity

completed

manual_order

Links

id

task_id

title

url

manual_order

Relationships

One household

↓

Many users

↓

Many tasks

↓

Many checklist items

↓

Many shopping items

↓

Many links

Priority Logic

Priority values

1

2

3

4

Within each priority:

Pinned tasks

↓

Manual order

No automatic sorting.

The user controls ordering.

Status Values

Active

Completed

Archived

Deleted records should not normally be removed from the database.

Soft deletion preferred.

Synchronisation

All editing should update in real time.

No refresh button.

No save button.

Auto-save after edits.

Conflict resolution:

Last write wins.

Simple.

Search

Version 1

Task title only.

Future

Notes

Description

Shopping items

Rich Text

Markdown support preferred.

Supported:

Headings

Bold

Italic

Bullet lists

Numbered lists

Links

No complex formatting.

Offline Behaviour

If internet unavailable:

Read existing tasks.

Queue edits.

Synchronise when connection returns.

If offline editing becomes too complex, it may be deferred to a future version.

Weather Integration

Only Outdoor tasks request weather.

Weather should update periodically.

Simple response only.

Weather states:

Suitable

Rain may make this task difficult.

Do not generate detailed forecasts.

Performance Targets

Application load

<2 seconds

Task opening

<300ms

Search

Instant

Dragging

60fps

Weather

<500ms

Security

HTTPS everywhere.

Passwords never stored directly.

Use Supabase authentication.

Database access restricted by Row Level Security.

Users only access their own household.

Error Handling

Graceful.

No technical error messages.

Example

Unable to save your changes.

Retry automatically.

Deployment

GitHub repository

↓

Automatic deployment

↓

Vercel

↓

Production

Every push to main deploys automatically.

Code Quality

Strict TypeScript.

Reusable components.

Small functions.

Meaningful names.

Avoid unnecessary abstraction.

Comment complex logic.

No dead code.

No duplicated components.

Folder Structure

/app

/components

/features

/hooks

/lib

/styles

/types

/public

Keep the project organised from day one.

Components

Task Card

Task Detail

Priority Section

Checklist

Shopping List

Link List

Weather Banner

Search

Navigation

Profile Menu

Button

Input

Modal

Each component should have one responsibility.

State Management

Keep simple.

Prefer React state.

Use server state where appropriate.

Avoid introducing global state libraries unless genuinely required.

API Design

REST or Supabase client.

Keep endpoints simple.

Avoid unnecessary complexity.

Testing

Unit tests for:

Priority ordering

Task movement

Pinning

Search

Weather state

Authentication

Integration tests for:

Create task

Edit task

Delete task

Archive

Restore

Synchronisation

Logging

Basic error logging.

No analytics required.

No advertising.

No tracking users.

Privacy first.

Future Integrations

Monzo

Budget checking

AI shopping assistant

Retail price comparison

Photo storage

Manual OCR

Equipment register

Recurring maintenance

Apple orchard management

Multiple properties

Calendar export

Widgets

Engineering Philosophy

When choosing between a clever solution and a simple solution, prefer the simple solution.

The application is expected to grow slowly over many years.

Readability and maintainability are more important than clever architecture.

The codebase should remain understandable by both human developers and AI coding assistants.

Every engineering decision should support long-term stability rather than short-term optimisation.